JQL for Jira & QA

Practical revision guide for Veeva QA Phase 1

Definition: JQL stands for **Jira Query Language**. It is used to search and filter Jira issues such as bugs, tasks, stories, and test-related issues using conditions.

Think of JQL as SQL-like searching for Jira issues. It searches issues; it does not create or modify them.

1. Basic syntax

A basic condition contains a field, an operator, and a value:

```
field operator value
project = QA
project = QA AND status = "To Do"
```

Replace the sample project key and values with those used in your Jira instance.

2. Common fields

Field	Meaning	Example
project	Project key	project = QA
issuetype	Issue type	issuetype = Bug
status	Workflow status	status = "In Progress"
priority	Configured priority	priority = High
assignee	Person assigned	assignee = currentUser()
reporter	Person who reported it	reporter = currentUser()
resolution	Resolution value	resolution IS EMPTY
created	Creation date/time	created >= -7d
updated	Last update date/time	updated >= -1d
labels	Issue labels	labels = regression
summary	Issue summary/title	summary ~ "login"
Sprint	Sprint field, if available	Sprint = "Sprint 5"

3. Operators

Operator	Meaning	Example
=	Equals	status = "Open"
!=	Not equal	priority != Low
>, >=, <, <=	Comparison for supported field types	created >= "2026-01-01"
~	Text contains/searches for term	summary ~ "login"
!~	Text does not match term	summary !~ "login"
IN	Matches one of several values	status IN ("Open", "Reopened")
NOT IN	Matches none of listed values	priority NOT IN (Low, Lowest)
IS EMPTY	Field has no value	assignee IS EMPTY
IS NOT EMPTY	Field has a value	assignee IS NOT EMPTY

Not every operator works with every field. Available values and comparison behavior depend on Jira configuration.

4. Logical operators

Operator	Meaning	Example
AND	Both conditions must be true	project = QA AND issuetype = Bug
OR	At least one condition must be true	status = Open OR status = Reopened
NOT	Excludes a condition	project = QA AND NOT status = Closed
Parentheses	Control grouping	project = QA AND (status = Open OR status = Reopened)

Memory tip: AND narrows results; OR broadens them. Use parentheses when mixing them.

5. Functions and date searches

Function/expression	Purpose	Example
currentUser()	Logged-in user	assignee = currentUser()
startOfDay()	Start of today	created >= startOfDay()
startOfWeek()	Start of current week	created >= startOfWeek()
startOfMonth()	Start of current month	created >= startOfMonth()
now()	Current date/time	updated >= now("-24h")
-7d	Seven days ago	created >= -7d

Function availability and accepted arguments can vary by Jira version, configuration, and installed apps.

6. QA-focused query examples

All bugs in project QA

```
project = QA AND issuetype = Bug
```

High-priority bugs

```
project = QA AND issuetype = Bug AND priority = High
```

Unresolved bugs

```
issuetype = Bug AND resolution IS EMPTY
```

Bugs assigned to you

```
issuetype = Bug AND assignee = currentUser()
```

Unassigned bugs

```
issuetype = Bug AND assignee IS EMPTY
```

Bugs created in the last seven days

```
issuetype = Bug AND created >= -7d
```

Bugs in progress

```
issuetype = Bug AND status = "In Progress"
```

Issues with "login" in the summary

```
summary ~ "login"
```

Your issues not in the Done category

```
assignee = currentUser() AND statusCategory != Done
```

Bugs newest first

```
issuetype = Bug ORDER BY created DESC
```

7. Jira + Xray examples

If Xray is installed and configured, test-related issue types may be searchable. Use the exact issue type names configured by your team.

```
project = QA AND issuetype = Test
project = QA AND issuetype = "Test Plan"
project = QA AND issuetype = "Test Execution"
```

Finding a Test issue is not the same as finding its latest execution result. Execution-result searches may require Xray-specific JQL functions or fields, depending on the edition and setup.

8. ORDER BY

```
issuetype = Bug ORDER BY priority DESC
project = QA ORDER BY created DESC
issuetype = Bug ORDER BY priority DESC, created DESC
```

ASC = ascending; DESC = descending. Priority ordering depends on the configured priority scheme.

9. Basic search vs. advanced search

Basic search	Advanced/JQL search
Uses graphical fields and menus.	Uses written JQL conditions.
Good for simple searches and beginners.	More flexible for complex conditions and functions.
Easy way to start.	Useful for precise queries and reusable filters.

In Jira issue search, switch to advanced/JQL mode where available, run the query, and save it as a filter if you have permission.

10. Common mistakes

• Using a project name instead of the actual project key.
• Searching for a status that is not in the workflow.
• Using summary = "login" when you mean to search for text; use summary ~ "login".
• Assuming status and resolution mean the same thing.
• Forgetting parentheses when mixing AND and OR.

11. Practice questions and answers

Question	Answer
1. Find all bugs in QA.	project = QA AND issuetype = Bug
2. Find High-priority Open bugs in QA.	project = QA AND issuetype = Bug AND priority = High AND status = Open
3. Find unresolved bugs assigned to you.	issuetype = Bug AND resolution IS EMPTY AND assignee = currentUser()
4. Find issues created in the last 7 days.	created >= -7d
5. Find your issues not in Done category.	assignee = currentUser() AND statusCategory != Done
6. Find bugs newest first.	issuetype = Bug ORDER BY created DESC

Final checklist: JQL meaning • fields • operators • AND/OR/NOT • currentUser() • date searches • ORDER BY • bug searches • Xray caveats.

Study tip: change one condition at a time and inspect the results. Sample values such as QA, Open, High, and Sprint 5 must match your Jira setup.